

Project information

Created on
March 27, 2024

Name	Last update
 resources	2 years ago
 .gitignore	2 years ago
 LICENSE	2 years ago
 README.md	2 years ago

 [README.md](#)

GSoC 2024 Proposal: Inkscape

Table of Contents

- [About Me](#)
- [Past Projects](#)
- [Journey with Inkscape](#)
- [Contributions](#)
- [Project](#)
- [Post GSoC Goals](#)
- [Availability](#)

About Me

Personal Information

- Name: Khushal Agrawal
- [Email](#)
- [Github](#)
- [Gitlab](#)
- [LinkedIn](#)
- RocketChat: skippr
- Location: India
- Timezone: IST(UST +5:30)

Education Information

- University : Indian Institute of Technology, Roorkee
- Major : Computer Science and Engineering
- Current Year : Sophomore (2nd year)
- Degree : Bachelor of Technology (B.Tech)

Background

I am Khushal Agrawal, a UG Sophomore majoring in Computer Science and Engineering at the Indian Institute of Technology, Roorkee. I like participating in various software development and tech-development endeavors, usually hackathons, CTFs, course projects, and projects at SDS Labs. SDS Labs is a student-run technical group that includes passionate developers and designers interested in various fields and involved in multiple software development projects that aim to foster a software development culture on campus. Being a part of this group has exposed me to different software development methodologies, tools and frameworks and helped me become comfortable contributing to an open-source project with multiple contributors.

Programming Languages

- **Languages:** C, C++, Python, Rust, Bash, HTML, CSS, Javascript, C#
- **Software Packages:** Git, Figma, Vim, MySQL, Visual Studio, CLion, Godot Engine, Unreal Engine, Unity Engine, Inkscape, Krita, Blender.
- **Others:** Object-Oriented Programming, Data Structure and Algorithms, Design and Analysis of Algorithms, Computational Geometry.

Why me?

I have plenty of experience contributing to open-source projects and a robust design sense, and I have been coding in Python, C and C++ for a long time. Moreover, I have experience working in SDS Labs, where we build production-level applications from scratch, and thus, I know the technical difficulties that might arise and how to deal with them.

Achievements

- **SASMO (Singaporean and Asian Schools Mathematics Olympiad):** Two times silver award winner
- **Finalists in Syntax Error 2023**, a 36-hr hackathon global hackathon organized by SDS Labs ([link](#))
- **Accepted in Winter of Code 2023**, a month-long programme internal to the members of our college meant to replicate the experience of GSoC. ([link](#))
- **Inter IIT Tech Meet 2023**, Gold Medal in Game Development Problem Statement

Past Projects

TinyRenderer | [Github](#)

- TinyRenderer is a series of graphic pipeline implementations written using OpenGL and C++ to show different graphics concepts such as lighting, transformations, camera, and model loading.
- It is implemented following the [learn-opengl](#) course.

RISCY | [Github](#)

- RISCY is a simple custom simulator of the 64-bit RISC-V pipelined processor implemented for a course project using C++.
- It can simulate simple RISC ISA, including complex optimisations such as operand forwarding.

NetStack | [Github](#)

- NetStack is a TCP/UDP network stack implemented above the Data Link layer using RAW sockets to expose the same API as provided by the transport layer with the same data format but faster.
- I managed to complete UDP, and for TCP, I implemented a three-way handshake for connect and disconnect, as well as the 'make packet' and 'send packet' functions.

Vector-DB | [Github](#)

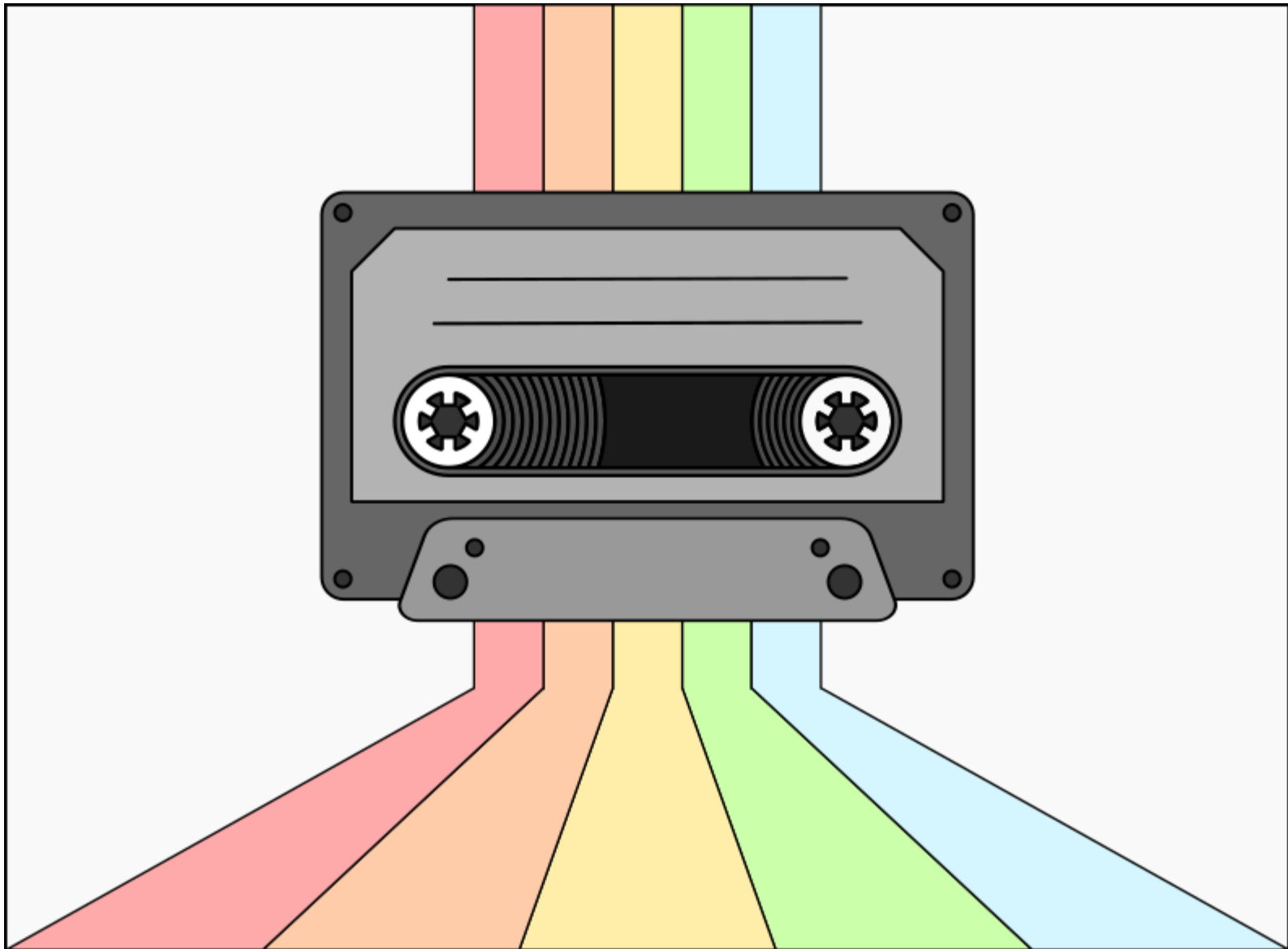
- Vector-DB is a project of SDS Labs that aims to create a vector-based database written in the Rust Programming Language.
- In this project, I implemented a wrapper around the RocksDB embedded database. I also implemented KD-Trees along with KNN algorithms to find the nearest points to an $O(N \log N)$ time entry in the vector space.

Saviour-Supreme | [Github](#)

- Saviour Supreme is a file transfer application developed in python that uses TCP and UDP sockets for safe yet fast file transfer over the network within our campus.

Journey with Inkscape

I was a very keen gamer when I was young. After playing many games throughout my childhood, I sometimes dreamed of ideas for games that would have been so fun and engaging. That was when I started learning game development to bring these ideas to life. I came across Inkscape as a versatile tool for developing assets for the games I used to create. I found Inkscape to be able to hide my poor artistic skills in any game art I made and, hence, stuck to it for a very long time. I use Inkscape mainly for pixel art and all other art aspects of the games I create (such as logos, title screens, backgrounds, and sprites). I will attach a few of my assets with a drive link [here](#). Below is an image I created using



Inkscape: In college, I learnt about open-source projects; one day, I stumbled upon Inkscape's Gitlab page, which made me realize Inkscape was also open-source. Since I had experience working with C++ and Python, I started taking up issues and attending the catchup meetings. Contributing to Inkscape was my closest experience to a professional work environment; it has helped me develop my communication skills (from talking to others in RocketChat to cleaner descriptive commit messages) and has made me a better developer (by trying to comprehend large codebases, fixing bugs, learning about CI runners, pipelines, SVGs, GTK4 and so much more). The Inkscape community kept me enthusiastic, as everyone was helpful and kind. I am suitable for this project because of my prior experience with C++ and Python and my familiarity with the Inkscape source code. Moreover, I have been a part of a development community (SDSLabs) for the past year, which has provided me with the experience of working on projects which have a developing codebase, and therefore know about the technical difficulties that arise due to old code that needs to be refactored or corrected as and when a bug pop up along with various other problems that arise.

Contributions

The best part about contributing to Inkscape was the proud feeling you get when a Merge Request has been merged, because now the changes you make will be affecting everyone who uses Inkscape in a positive way.

- Add icons to export dialogs (Status: **Merged**) [!6166](#).
- Fixes crash on incomplete closing of parenthesis (Status: **Merged**) [!6167](#).
- Fixes the snapping of ellipse allowing user to draw accurate circles (Status: **Merged**) [!6199](#).
- Fixes crash for star ratio set to 0 (Status: **Merged**) [!6221](#).
- Fixes setFinal crashing in Path in lib2Geom (Status: **Merged**) [!120](#).
- Fixes unwanted cusp node creation in Node tool (Status: **Merged**) [!6285](#).
- Adding extra Node tool delete behavior (Status: **Open**) [!6288](#).
- Fixes the snapping on Ctrl+Shift (Status: **Closed**) [!6154](#). Closed because this would over complicate the Arc Tool

Along with coding, I have involved myself with discussions about applicability and reproducibility of issues. Also, I have reported the following issue myself:

- [10168](#) in inbox

Project: Improving Workflow of Node and Bezier Tool

This project would improve the workflow of editing of geometry of paths. Combining strengths of Pen tool and Node tool with modifiers and new behaviors. This improvement should benefit professional users but also beginners. A good example of this implementation is in Blender's new Pen tool.

Initial Research (Blender's Pen tool (Curvature Pen))

Features:

- The curvature pen tool in Blender is similar to the pen tool in Adobe Illustrator or Bezier Curves in Inkscape, making it familiar for those with experience in these programs.
- The tool offers a simple efficient way to create and edit geometry, allowing for precise control over the shape and curves of the objects.

- By combining the curvature pen tool with other workflows, such as scaling or adjusting positions, users can achieve even more complex and versatile results.
- The tool provides various shortcuts and actions, such as double-clicking to change point types or holding Shift to break handles, which enhance the speed and flexibility of the workflow
- The ability to add, delete, and adjust points on the fly gives users full control over the geometry, making it easy to iterate and refine their designs.
- Some important features of this tool that could be implemented in Inkscape are:
 - Double clicking on a point can change it to a sharp point or smooth out broken handles.
 - Holding Alt and changing the size of a handle can create variations in the geometry
 - Holding Ctrl and clicking can add new points, while Ctrl-clicking on a point can delete it.

Resources:

- <https://www.youtube.com/watch?v=iwfgG1efL4U>
- <https://docs.blender.org/manual/en/latest/modeling/curves/tools/pen.html>
- <https://www.youtube.com/watch?v=APxkUd7OAAk>
- <https://inkscape.org/doc/keys.html#idm1890>

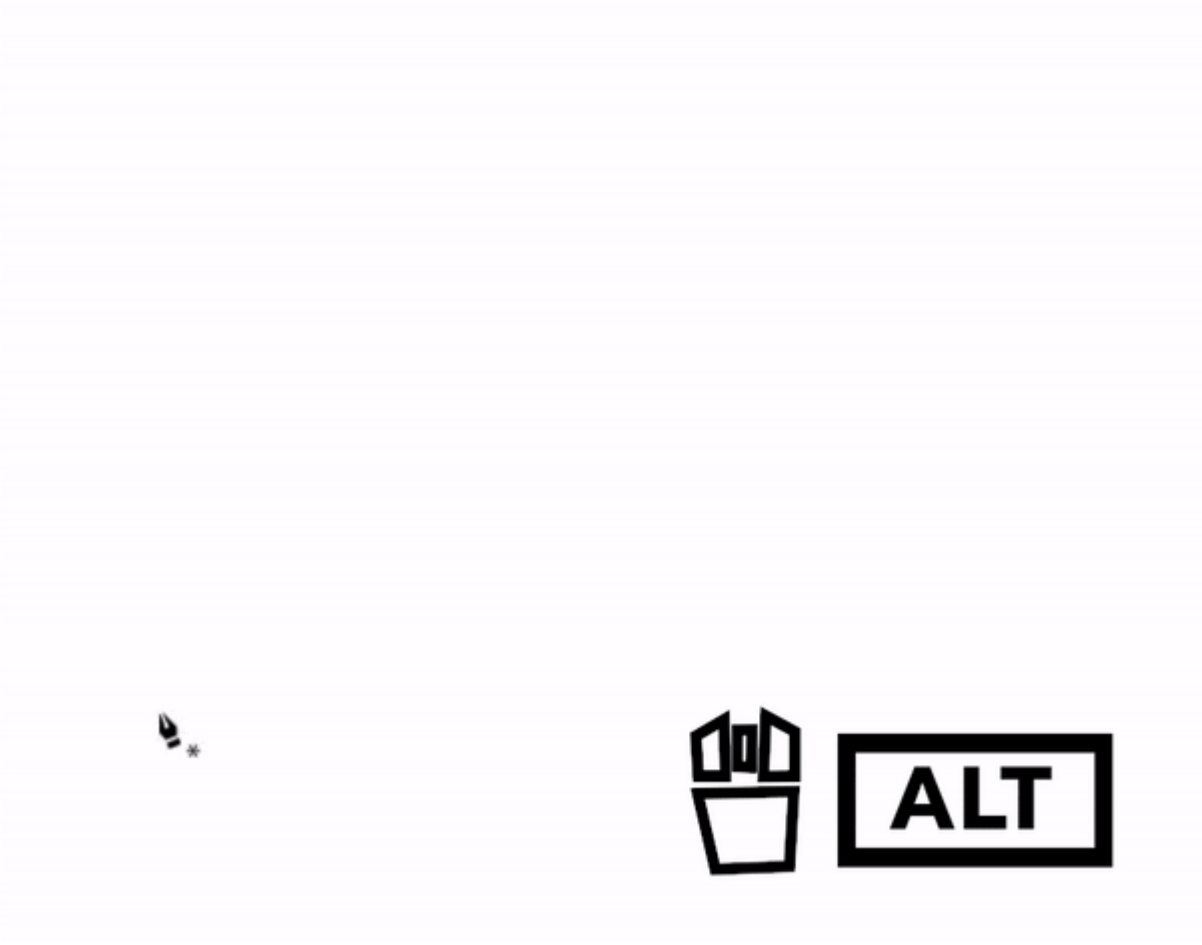
Bezier Tool

Features:

To have an efficient and intuitive workflow the Bezier tool should have the following list of features:

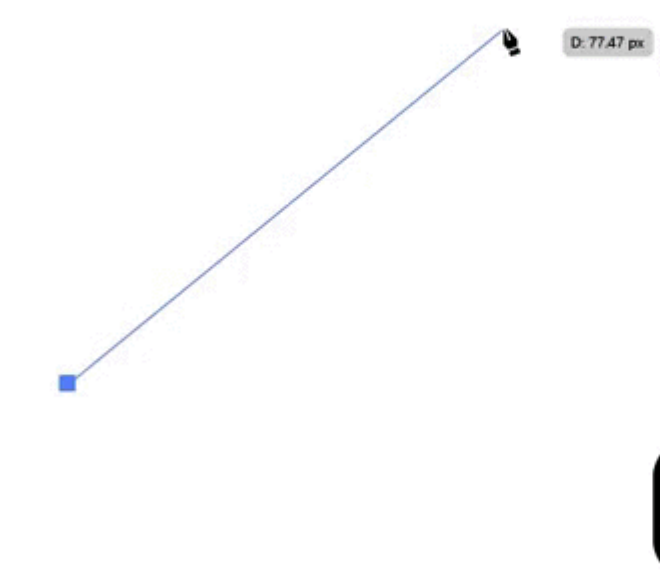
1. Show all nodes of the paths and give access to the user to edit them while they are still in the bezier tool.

2. Allow the user to quickly toggle to node tool without leaving bezier tool (by holding the ALT key)



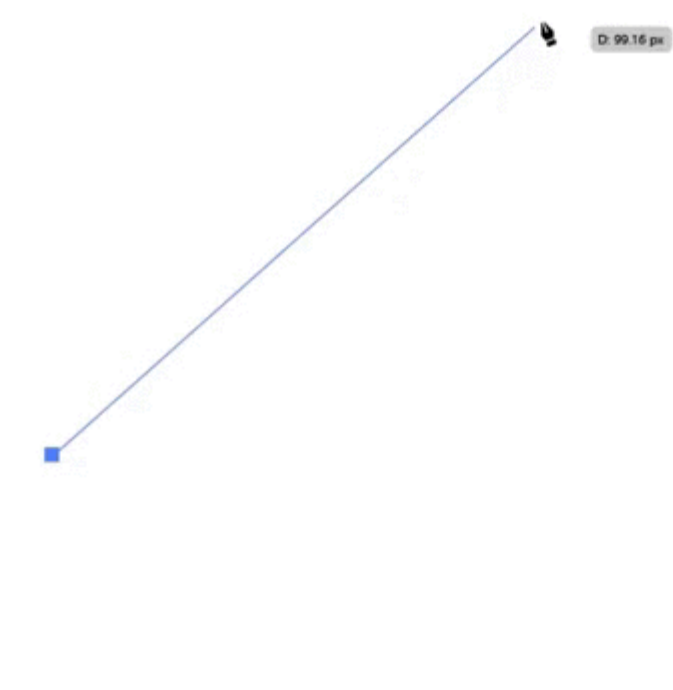
3. Allow the user to control the handle length individually while still in the tool (by holding the ALT key while changing length).

BEZIE CURVES TOOL - CHANGE LENGTH OF ONE SIDE OF HANDELS



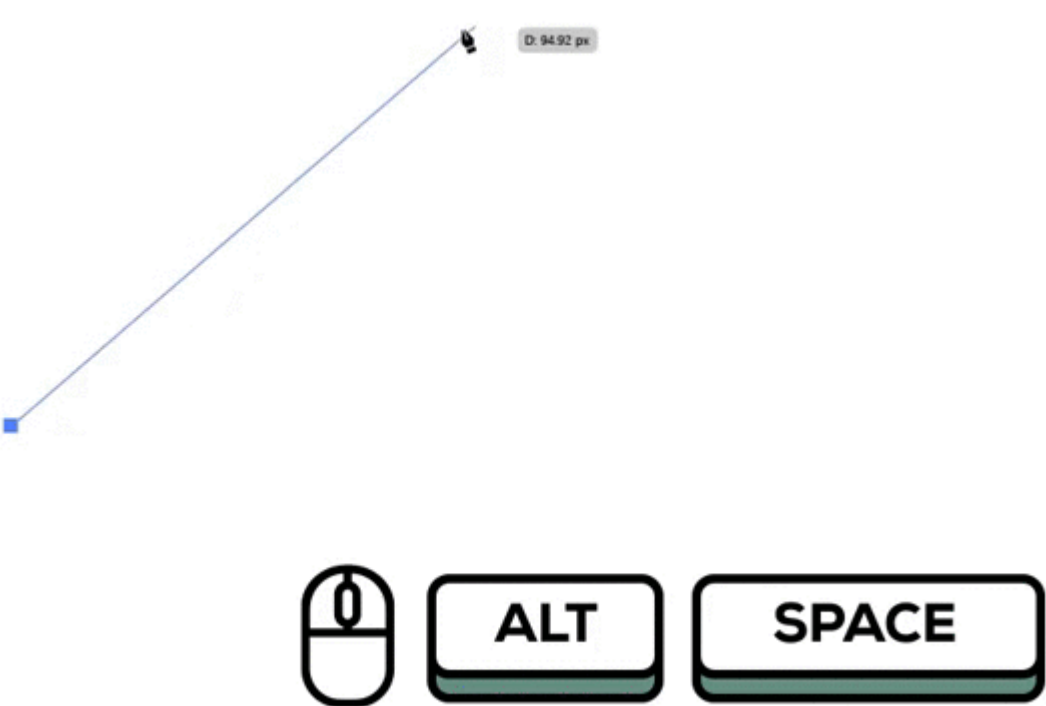
4. Allow the user to delete one side of a handle (without using SHIFT + L) (preferably by clicking on the last node again).

BEZIE CURVES TOOL - CONVERT LAST NODE TO CORNER POINT



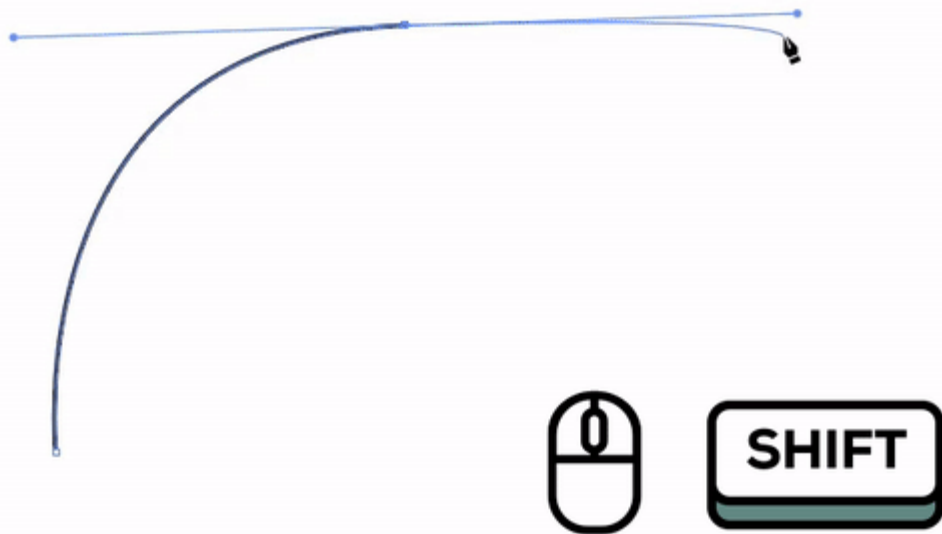
5. Allow the user to move a node after you have put it down (using ALT+SPACEBAR or SHIFT + SPACEBAR).

BEZIE CURVES TOOL - MOVE NODE AFTER YOU STARTED DRAGING



6. Allow the user to SHIFT + Click on the last handle to create corners

BEZIER TOOL - BRAKING HANDELS (CORNER POINT)



Implementation:

Implementation of this tool involves working mostly with the Pen Tool and the Pencil Tool.

1. Currently while drawing, the first node, the last node with its handles, and the second last node are visible in the pen tool. Pen tool contains an array of `CanvasItemPtr<CanvasItemCtrl>` these represent the visual nodes that are created while drawing the tool. These are also part of `SPDrawAnchor` that are used in the `FreehandBase` class to represent start and end anchors. `SPDrawAnchor` has a function called `anchorTest` which makes the anchor active, if the mouse hovers over it. To show all the nodes in the path, I will create a new `Vector<SPDrawAnchor>`. Every time a new node is appended to the `green-curve` I will append to the vector and set the new handle to the position of the new node and set it to be visible. Using an anchor will be beneficial in the other functionalities that are needed to be implemented below.
2. **Note:** The functionality of this feature is still under discussion. There are two possible ways to implement this tool:
 1. When the `ALT` key is pressed the `red-curve` stops displaying and the previously created nodes can be moved around and their handles manipulated. To implement this functionality, we can add two modes to the pen tool, the `NORMAL` and the `NODE` modes, when the user presses the `ALT` key (and the user is not dragging the mouse) we switch modes. In this new mode, we will reset the `red-curve` and check for the `anchorTest` function in the `Vector<SPDrawAnchor>` we created earlier. If the mouse is dragged while any of the anchors are `Active` then we know which node the user is trying to manipulate, we rewrite the `_redrawAll()` function to create the `green-curve` based on the vector of anchors. Whenever the user drags a `DrawAnchor`, after release the `Vector<SPDrawAnchor>` will be updated and the `_redrawAll()` function is called. While dragging an anchor, we can draw a `red_bpath` from the previous anchor in the vector to the next anchor in the vector, giving user feedback of the new curve that will be formed. This allows the user to move nodes. When the user clicks on an anchor, (while in `NODE` mode) the handles of that particular node would be displayed, by fetching the equivalent segment of the `green-curve`, allowing the user to update these handles. The implementation would be similar to moving nodes.
 2. When a specific key is held (`N` key was discussed on [chats](#) but requires confirmation from the UX team). The current curve is changed to a path and the entire tool is switched to the node tool allowing the current path to be edited, on releasing the key, the curve continues to behave like it would in the node tool. This could be done by finishing the curve, and storing the last node in the pen tool, calling the node tool from the pen tool along with the selection as the given path. A variable in the node tool holds the current state of the tool, when the key is released it returns to the pen tool, and the pen tool uses the last node of the path and continues drawing the curve (the added points are then attached to the previous path on completion).
3. When the user is in the `MOUSE_DRAG` mode and `Shift` key is held, it detaches the two handles and lets them behave individually, a slight modification of this code, when the `ALT` key is pressed, such that the individual handles maintain their lengths (independent of each other) but are collinear. ([here](#))
4. This is where the use of `SPDrawAnchor` will be helpful, if the last anchor in the vector of draw anchors is active and a mouse click is registered, then the existing function `_lastpointToLine()` can be called to create a corner on the last node.
5. To allow the user to move a node after it has been placed, we can use pre-existing functions of the pen-tool, the pen-tool contains a function called `_lastpointMove()` which takes a `gdouble x` and `gdouble y` as its parameters and converts it to global coordinates. We can overload the function to take a `Geom::Point` as a parameter (the global coordinates directly). The positions of the `red-curve` (the curve that displays the current undrawn segment) are held in `p_array`. When the user presses `ALT+SPACEBAR` we note the previous length and direction of the vector from `p_array[3]` to `p_array[4]` (representing the forward control handle) and update the value of `p_array[4]` according to the current mouse position, and the value of `p_array[3]` can be calculated according to previously stored values. The values of the handles and the control points (`c11`, `c10`, and `ctrl[4]`) should also be updated according to these values in the `_setCtrl` function.
6. This implementation already exists in the current state of the pen-tool ([here](#)). When the `Shift` key is pressed while dragging the handle, the position of the other handle remains unchanged creating a corner.

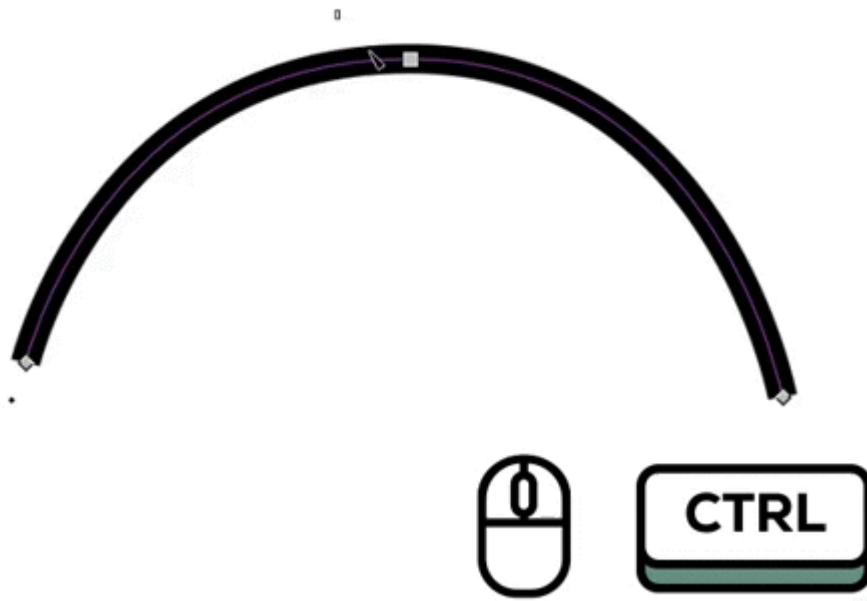
Node Tool:

Features:

Similarly, to have an efficient and intuitive workflow the Node tool should have the following list of features:

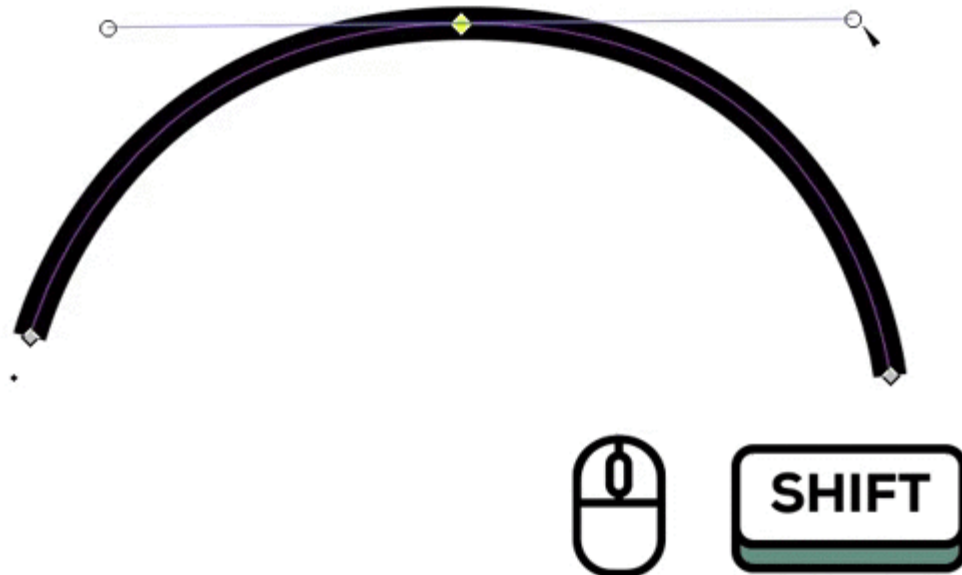
1. Allow the user to snap to pre-defined angles when the handle is dragged while holding CTRL.

NODE TOOL - SNAPPING TO ANGEL



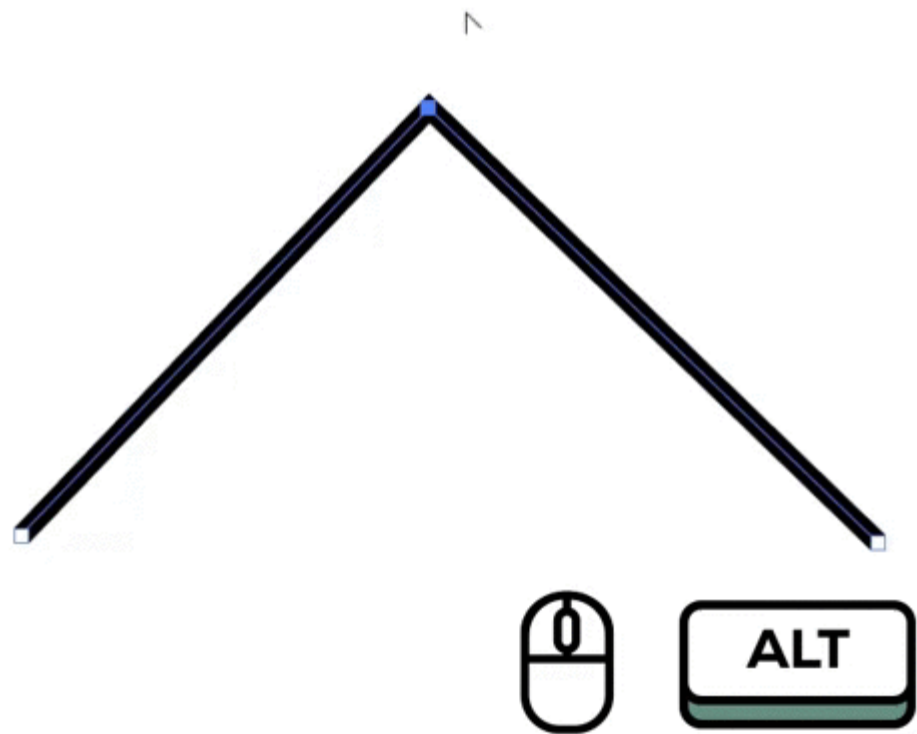
2. Allow the user to break handles (creating corners) by holding SHIFT while dragging a handle.

NODE TOOL - BRAKING HANDELS (CORNER POINT)



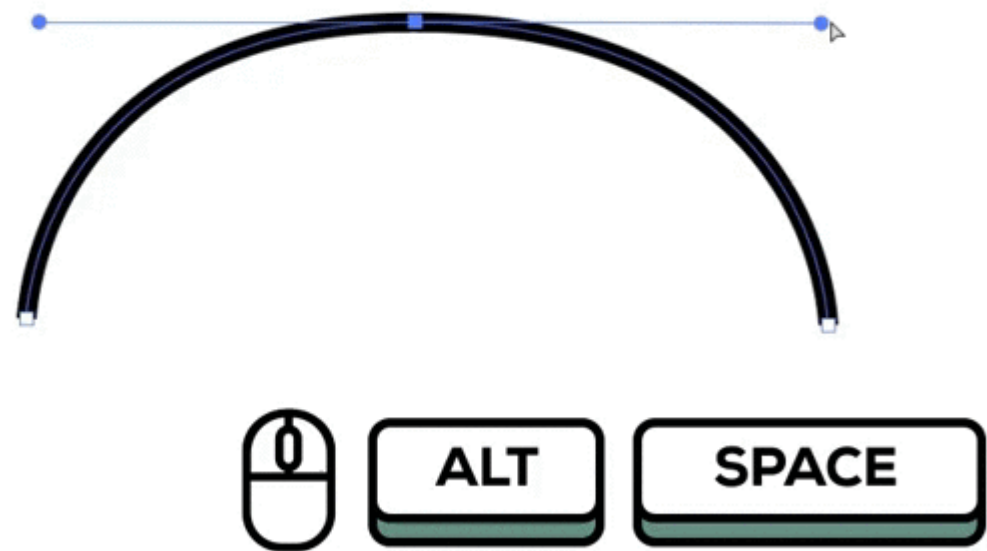
3. Allow the user to link the two bezier handles and make them the same length by pressing and holding the ALT key while dragging.

NODE TOOL - SAME LEGTH OF BEZIER HANDELS WITH LINK

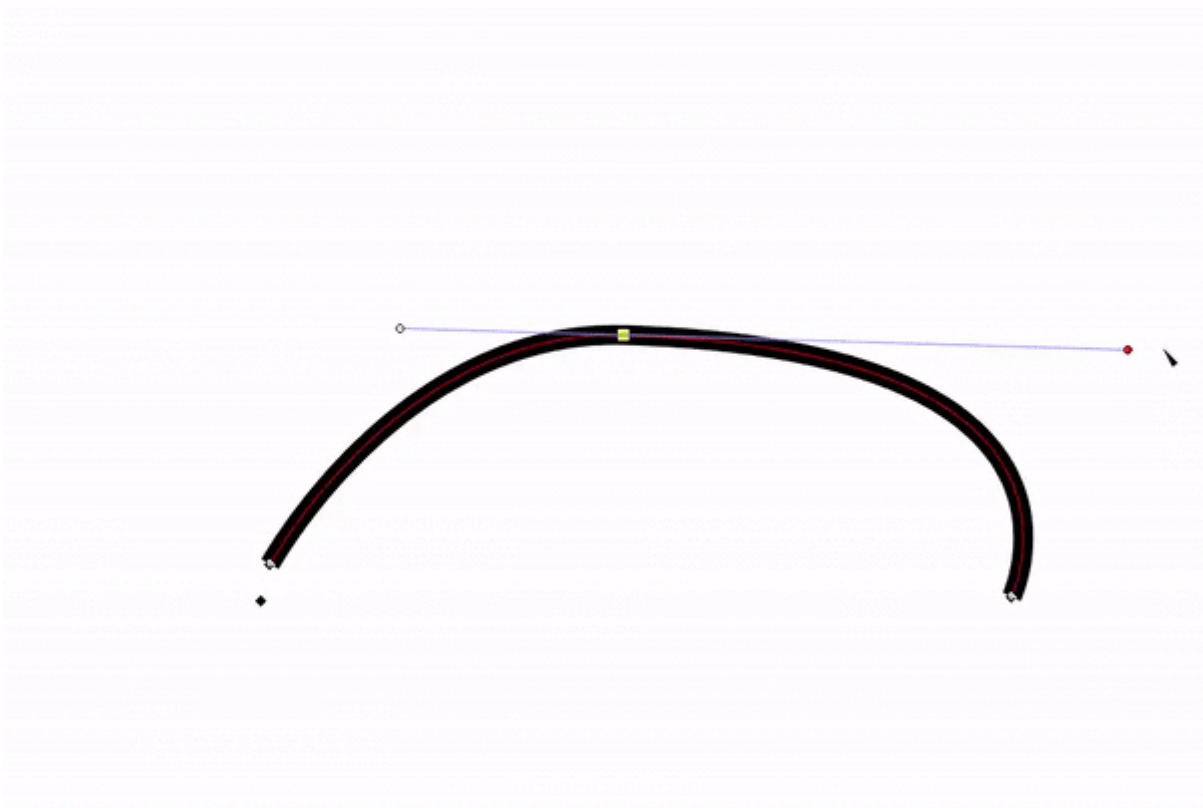


4. Allow the user to move the node after they started dragging the handle (using ALT + SPACEBAR).

BEZIE CURVES TOOL - MOVE NODE AFTER YOU STARTED DRAGING

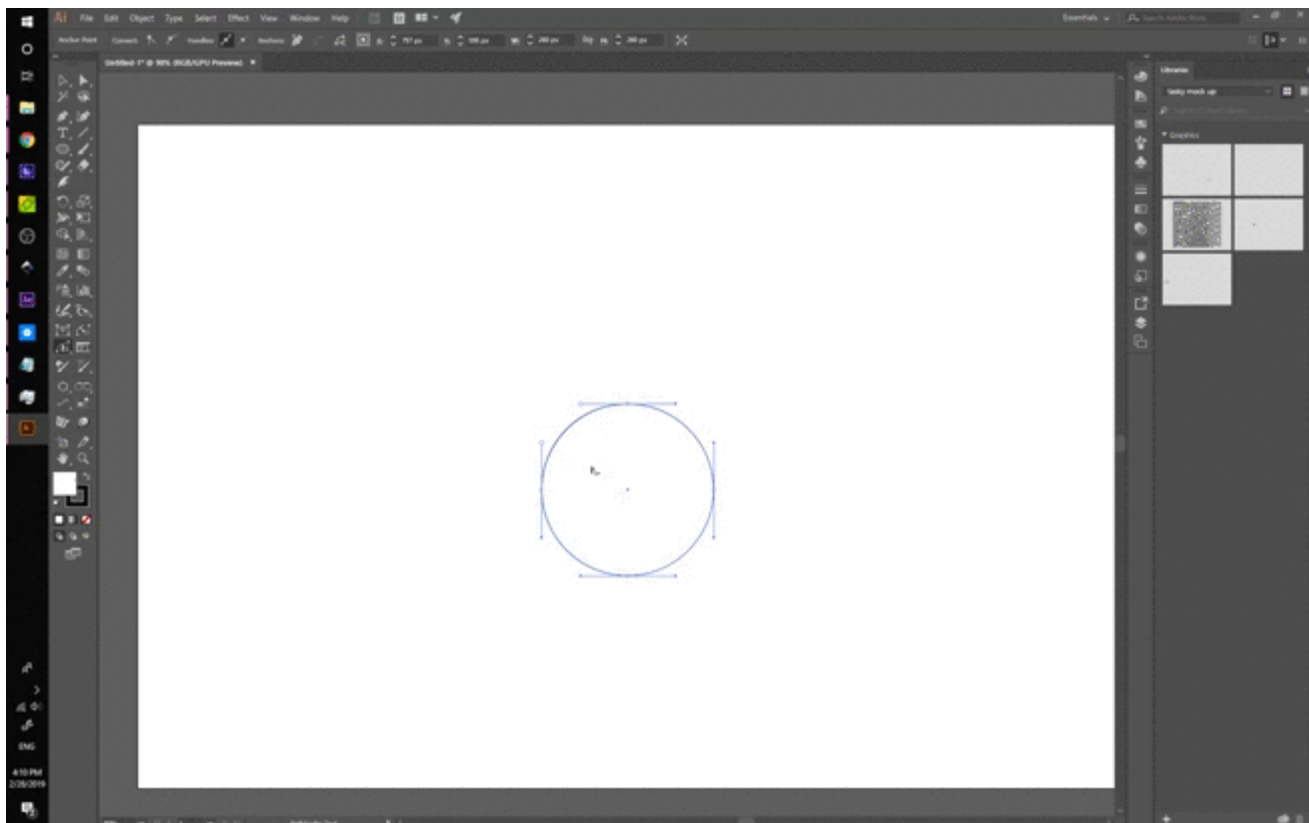


5. Allow the user to change the size of one handle without pressing hotkeys (the handles should be linked normally but the change of size



should be individual).

6. Allow the user to select and manipulate multiple nodes together in the following manner:



1. **Mode 1:** All the linked handles move together relatively linked to the position of a handle the user is manipulating.
2. **Mode 2:** All handles continue in their original direction
3. **Mode 3:** All handles continue in the reverse direction.
7. Add a new icon for every node type to improve tool clarity

Implementation:

Implementation of this tool involves working with the `node-toolbar`, `node-tool` and `node` files.

1. This functionality is already present in the current Node Tool; they are implemented using the `Handle::dragged` function in `node.cpp`. When ctrl is held while dragging, the `snap_increments` are retrieved from the preferences and the `snap_pos` is calculated by using the `Geom::constrain_angle` method. If the `snap_pos` turns out to be large enough, the handle is snapped to this new position after the `_pm.update()` method is called at the end of the function. ([here](#)).
2. By default, the handles of a node are collinear if the node is of the type `NODE_SMOOTH` or `NODE_SYMMETRIC`, to break these handles, we just need to convert the `NodeType` to `NODE_CUSP`, in this node type the other handle is completely ignored while moving the current handle. Again in the `Handle::dragged` function we need to check for the `held_shift(event)` and set `_parent` (Node) type using: `_parent->setType(NODE_CUSP, false)`.
3. To allow the user to link handles by holding the `ALT` key, it would be done simply by changing the `NodeType` of the the `Node` to `NodeSymmetric` until the `ALT` key is held and then returning the node to `NodeSmooth` after the `ALT` key is released. Currently on pressing `Shift + Y` the Node tool permanently changes a node's type to `NodeSymmetric`. I have attached below a pseudo-code to achieve the same implementation:

```
void Handle::dragged(Geom::Point &new_pos, MotionEvent const &event)
{
    ...
    // with Alt, link both the handles to be of the same length
    if (held_alt(event)) {
        other()->setRelativePos(-relativePos());
    }
    ...
    _pm.update();
}
```

Currently when the ALT key is held, it fixes the size of the handles of the node. To prevent the loss of this functionality, upon discussing with the UX team, I will assign this feature to another shortcut. I would implement this as a toggle, (maybe using `SHIFT+E`) to implement it, I would add a variable `fixed_lengths` to the `Node` class and toggle it when `Shift+E` is pressed while hovering over a handle, I will add the sample code below:

```

...
bool Handle::_eventHandler(Tools::ToolBase *event_context, CanvasEvent const &event)
{
    bool ret = false;

    inspect_event(event, [&] (KeyPressEvent const &event) {
        switch (shortcut_key(event)) {
            ...
            case GDK_KEY_a:
            case GDK_KEY_A:
                _parent->fixed_lengths = !_parent->fixed_lengths;
        },
        ...
    });
    return ControlPoint::_eventHandler(event_context, event);
}

...

void Handle::dragged(Geom::Point &new_pos, MotionEvent const &event)
{
    ...
    // with Alt, link both the handles to be of the same length
    if (held_alt(event) && !_parent->fixed_lengths) {
        other()->setRelativePos(-relativePos());
    }
    if(_parent->fixed_lengths) {
        new_pos = parent_pos + Geom::unit_vector(new_pos - parent_pos) * _saved_length;
        snap = false;
    }
    ...
    _pm().update();
}
...

```

4. Since the handles contain pointers to their parents (`Node`) called `parent` . When the `ALT` and `SPACE` keys are simultaneously pressed while dragging the handle, `Handle::dragged` function is being called. The new position of the parent can be calculated relative to the handle and the function `_parent->dragged(new_pos, event)` can be called to move the parent `Node` according to the new position. Currently the `Handle` holds a variable called `_saved_length` that is updated at the end of each cycle, so that the next time, it acts as the previous saved length. Similarly, I will add a variable called `_saved_dir` that will be a unit vector from the parent position to the handle position. When the relevant keys are pressed while dragging the handles, the position of the parent would be calculated using `_saved_dir` , `_saved_length` and `new_pos` of the handle.
 5. By default, if a node is of the type `NODE_SMOOTH` the handles are independent of each other, they just have to be collinear. This behavior is already implemented very effectively in the current state of Inkscape.
 6. In the current state, Inkscape allows the user to move multiple handles together relative to their world positions (i.e move each handle equal distance in the same direction). To implement the modes and allow manipulation of multiple handles also. The first step to take would be adding these methods in the `node-toolbar` , the icons and their descriptions would be added by discussing with the UX team. The steps followed for this would be to first edit the `toolbar-node.ui` to add these buttons to the toolbar, then `node-toolbar.cpp` would link these toolbar buttons to their respective functions and implementations.
- As for the handling of the handles according to these modes, currently a `Handle` has a `ControlPoint` , we would need to change this to a `SelectableControlPoint` , a hierarchy needs to be implemented to simultaneously manipulate all handles (`MultiHandleManipulator`) similar to the hierarchy and implementation of `MultiPathManipulator` and `PathManipulator` for the node tool. The implementation of `Mode2` and `Mode3` would be trivial, for the implementation of `Mode1` the angle rotated relative to the initial position and the distance changed can be calculated, the same can be applied to all other handles keeping consistency with their node types (i.e for example changing the length of the the other side handles and keeping them collinear if the node type is `NODE_SYMMETRIC` and so on).
7. The node handles can be after this [MR](#) was approved. The visual appearance of these handles can be changed by changing the `share/ui/node-handles.css` file. Each of the node types has a `css` that describes its shape, size and color. To change the shapes of each node type, I need to separate the classes and add separate shapes for each. ([here](#))

Use Cases:

- The project aims to rework the workflow of Node and Bezier Tools to improve and significantly speed up the work of professional Inkscape users by removing the time spent toggling between Bezier and Node tools. For example:
 - Pressing the `ALT` key while in the pen tool allows the user to manipulate previously created nodes and handles (like the node tool).
 - In both the tools pressing `ALT + SPACE` while dragging a handle allows the user to move the node instead, reducing the time spent switching between a node, and its handles.
- This rework also helps beginner users of Inkscape as it is much more intuitive than the current UX of the Node and Bezier tools, and the lowered learning curve due to this intuitiveness would help keep them motivated to learn the tool.
 - These features make the Node and Pen tools incredibly similar to each other, reducing the number of tools a user needs to grasp

before being efficient in using the application.

- These tools also become much more similar to the Bezier-Node tools from Blender and Illustrator, making migration easy for users from respective platforms.
- Several same shortcuts in these different tools would perform the same functionality increasing intuitiveness.

Stretch Goals:

I plan on creating added support for editing arc segments in the node editor similar to: [nan.fyi](#)

There have been previous discussions on this, Adam Bellis has also opened an [issue](#) where he completely describes the UX of the desired feature that I wish to follow. There has also been a previous [implementation](#) that was not successfully merged due to incorrect behavior of the end points of the arc-handles. I plan to build upon the work done in this issue and add this functionality to the node tool.

Timeline

Date (from and to)	What I plan to do
Pre GSOC Period	
2nd April to 1st May	Solve issues more focused towards the Node and Bezier Tools to have an even better understanding of the codebase.
GSOC Period	
1st May to 26th May [Community Bonding]	Discussion with mentors about the project and finalize the design choices with the UX team. Talk to other developers and designers to know if they have some recommended changes to the design or implementation of the Node and Bezier tools. Learn more about the code base and relevant areas I would be working upon.
Coding Period Begins	
27th May to 2nd June	Start working on the Node tool. Implement the feature of linking bezier handles so that their lengths change symmetrically (3) and making the default behavior to change the length of just one side (5).
3rd June to 9th June	Implement the feature of allowing selection and manipulation of multiple bezier handles (6)
10th June to 16th June	Implement unique shapes for every node type (7) and the feature to allow moving nodes after you have started dragging handles. (4)
17th June to 27th June	Implement the feature where holding CTRL snaps the handles to fixed angles. (1) Implement the feature to allow breaking of handles for corner points (2)
28th June to 5th July	Provide the Inkscape Developers and UX team the new version of the Bezier tool to test and give feedback and recommendations. Implement any additional feature request or changes that I might receive as feedback. Update the modified shortcuts and functionalities to the Inkscape Wiki
Mid Term Evaluation and Report (6th to 9th July)	
10th July to 21st July	Start working on the Bezier tool. Implement the feature to show all bezier handles while still in edit mode. (1) Implement the quick toggle to node tool feature (2)
22nd July to 28th July	Implement the feature to control the length of the newly created handle. (3) Implement the feature of deleting one side of the handle. (4)
29th July to 4th August	Implement the feature to add a key modifier to move a node after you put it down. (5) Implement the feature to allow breaking of handles without switching tools. (6)
5th August to 15th August	Provide the Inkscape Developers and UX team the new version of the Bezier tool to test and give feedback and recommendations. Implement any additional feature request or changes that I might receive as feedback. Update the modified shortcuts and functionalities to the Inkscape Wiki.
16th August to 20th August	This period will be for the rigorous testing of my code to guarantee no errors are left behind, and other miscellaneous activities such as code cleanup, documentation, etc.
20th August	Final Evaluation
21st August onwards	Work on Stretch Goals

Post GSoC Goals

I am incredibly passionate about contributing to open-source organizations. I have been able to overcome the initial learning curve of contributing by getting familiar with the codebase, Gitlab and RocketChat. I plan to contribute to Inkscape even after completing GSoC by adding new features (I plan to implement the stretch goals if they are not finished within the GSoC program duration), reporting bugs, maintenance and solving bugs. I will also solve any other issues of maintenance needs that come up in the future regarding the Node and Bezier Tools.

Availability

I am willing to dedicate more than 30 hours a week. The college's summer vacations are scheduled from mid-May to July. Therefore, there would be no classes for the first two months of the program's coding period. My college will reopen in mid-July. I can work more than 20 hours a week during this time. I generally work from 2 PM to 3 AM IST (4:30 AM to 5:30 PM Eastern Time), although I am willing to adjust my working hours and timings if required.

Besides this project, I have no commitments/ vacations planned for the summer. I shall keep my status posted to all the community members every week and maintain transparency in the project.

I am dedicated to Inkscape and do not plan to submit a proposal to any other organization.